Structured output (a checked answer, not a hope) — Hints

Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The idea: a form with locked fields
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea: a form with locked fields

Basic Version

A Pydantic model is a class that describes a shape: which fields exist, and what type each one is. You write it once, like a form:

```
from pydantic import BaseModel


class ExtractedInfo(BaseModel):
    name: str | None
    intent: str | None
```

`ExtractedInfo.model_validate(some_dict)` checks a dict against that form. It gives you back an object when the dict fits, and raises `ValidationError` when it doesn't. Try it first with a plain dict — no API call, no cost.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Two kinds of field do most of the work:

- `str | None` — the field **must** be there, but `None` is an allowed answer. A plain `str` would force the model to invent a name when the message has none (the Structured output topic above).
- `Literal["low", "normal", "urgent"]` (from `typing`) — only those exact values pass. `"very"` or `"Urgent"` raises `ValidationError`.

Then the API call: `client.beta.chat.completions.parse(model=..., messages=..., response_format=ExtractedInfo)`. It sends your class as a strict schema, and runs `model_validate` on the reply for you.

Difference between Basic and Intermediate: Basic shows the form and the hand check. Intermediate adds the two field types a real schema needs, and the one call that makes the model fill the form.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define ExtractedInfo: name, intent, urgency
check a good dict and a bad dict by hand
call .parse() with response_format=ExtractedInfo
message = the first choice's message
if message.refusal: print it
else: print message.parsed and its fields
```

The part that's new — reading the answer:

```
message = completion.choices[0].message
if message.refusal:
    print(f"The model refused: {message.refusal}")
else:
    info = message.parsed
    print(f"name: {info.name}, urgency: {info.urgency}")
```

`message.parsed` is already an `ExtractedInfo` object — use `info.name`, not `info["name"]`.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
function extract_info(text) -> ExtractedInfo | None:
    try: completion = .parse(system prompt + text, ExtractedInfo)
    except ValidationError: return None
    message = the first choice's message
    if message.refusal: print it, return None
    info = message.parsed
    if info.name is None and info.intent is None: return None
    return info
```

Tell the model in the system prompt to use `null` instead of guessing — the `str | None` field makes that possible, the prompt makes it happen.

Returning `None` for "nothing found" gives the caller exactly two cases: an object it can trust, or `None`. Run it on a clear message, a message with no name, and `"the sky is blue"`, then compare against the Solution.

Last, let the user type the text: loop on `input("You: ")`, stop on `/quit`, and when a line `.startswith("/extract ")`, pass `line.removeprefix("/extract ")` to `extract_info()`.

Difference between Basic and Intermediate: Basic reads one answer inline. Intermediate wraps it in `extract_info()`, which turns a refusal, a bad reply, and an empty answer into one clear `None` — the exact shape the Build Task's `get_structured_reply()` needs.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution